Week 9 — Orchestrated Batch Scoring

Course: DS5619 Machine Learning Systems Operations
Student ID: `142301024`
Seed: `2083192409` (salt: `week09`)

What Was Implemented

In this lab, I built a lightweight DAG (Directed Acyclic Graph) workflow orchestrator in `src/mini_orchestrator.py` to schedule and execute batch inference tasks with dependency management and retry handling.

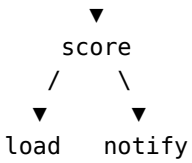
The three functions implemented are:

1. **`topological_order(tasks)`**:
   - Implements Kahn's algorithm to resolve task execution order so every task only runs after all of its dependencies finish.
   - Computes in-degrees (number of dependencies) and maintains an adjacency list.
   - Validates that every declared dependency actually exists in `tasks`, raising a `ValueError` for unknown tasks.
   - Repeatedly pops tasks with in-degree 0 and decrements downstream dependency counts.
   - If unplaced tasks remain when in-degree 0 tasks are exhausted, it raises a `ValueError` indicating a dependency cycle.
2. **`execute_with_retry(fn, retries=3, delay_sec=0.1)`**:
   - Executes a task function with automatic retry logic for transient errors.
   - If the function raises an exception, it sleeps for `delay_sec` and tries again, up to the maximum retry count.
   - Returns the result as soon as an attempt succeeds, tracking the number of attempts taken. If all retries are exhausted, it re-raises the final exception.
3. **`run_dag(tasks)`**:
   - Resolves the execution order by calling `topological_order(tasks)`.
   - Executes each task sequentially using `execute_with_retry`.
   - Passes upstream task outputs to downstream tasks as input arguments, and collects execution status and attempt counts into a summary dict.

Results

Running the pipeline (`python src/run_pipeline.py`) executed the 4-task batch scoring workflow across 60 CCTV images:

```
extract
   |
```



1. Execution Summary (`dag_run_summary.json`):

- **Execution Order:** `["extract", "score", "load", "notify"]`
- **Task Results:**
  - `extract`: status = `success` (1 attempt)
  - `score`: status = `success` (1 attempt)
  - `load`: status = `success` (1 attempt) — wrote `batch_report.json`
  - `notify`: status = `success` (3 attempts) — simulated an unreliable webhook that failed on attempts 1 and 2, but succeeded on attempt 3.

2. Batch Scoring Report (`batch_report.json`):

- Successfully processed all **60 images** across both camera profiles (30 daylight, 30 lowlight).
- Recorded total detected bounding boxes and vehicle classifications per image.

Key Takeaway:

The DAG orchestrator correctly decoupled task execution: even though the `notify` task failed twice before recovering, the sibling `load` task completed independently without delay, demonstrating branch isolation and automated failure recovery.